

Rapport Projet Fabrique Lunettes Java avancée

M1 MIAGE - S8

2025/2026

Achraf RAOUF - Ines JUNG - Flora BOLZON - Riad BELAZOUGUI

Date : 22 Mai 2026

Dépot : <https://github.com/gayariad/fabrique-lunettes.git>

Sommaire

1. Introduction.....	4
1.1. Contexte et objectif du projet.....	4
1.2. Présentation de l'équipe et rôles de chacun.....	5
1.3. Méthode de travail.....	6
2. Analyse du cahier des charges et planification.....	6
2.1. Lecture et compréhension du sujet.....	6
2.2. RoadMap initial.....	7
2.3. Recherches préliminaires et sources.....	8
MQTT et Publish/Subscribe.....	8
Eclipse Paho, Client MQTT Java.....	8
JavaFX 21.....	8
Maven multi-module.....	8
Ping-pong scheme.....	8
3. Architecture du système.....	9
3.1. Vue d'ensemble.....	9
3.2. Choix architecturaux fondamentaux.....	10
Pas de Spring (contrainte du sujet).....	10
Format de sérialisation custom (contrainte du sujet).....	10
Structure Maven multi-module.....	11
3.3. Architecture backend :.....	11
3.4. Architecture client JavaFX.....	12
3.5. Protocole de communication MQTT.....	14
4. Les dépendances : comment et pourquoi.....	16
Structure Maven multi-module :.....	16
Eclipse Paho MQTT Client :.....	16
SLF4J API.....	16
Logback Classic.....	17
JUnit 5 Jupiter.....	17
Mockito.....	17
Maven Surefire Plugin.....	18
Bibliothèque fabricant.....	18
JavaFX 21.....	18
Gson.....	18
Maven-shade-plugin.....	19
Javafx-maven-plugin et jlink.....	19
5. Implémentation détaillée.....	20

5.1. L'Usine / cœur du backend.....	20
5.2. Le Serveur et le callback MQTT.....	21
5.3. Le client JavaFX.....	22
5.4. Le protocole de sérialisation custom.....	28
5.5. Gestion des erreurs.....	28
6. Tests.....	29
6.1. Stratégie de test.....	29
6.2. Tests backend / UsineTest :.....	29
6.3. Tests backend / ServeurCallbackTest.....	30
6.4. Tests client / CommandeModelTest et LivraisonModelTest.....	31
6.5. Exécution et intégration CI.....	31
7. CI/CD.....	32
8. Conformité au cahier des charges.....	34
8.1. Exigences fonctionnelles.....	34
8.2. Exigences non-fonctionnelles.....	36
8.3. Exigences de livrable.....	37
9. Réponses aux questions du sujet.....	38
9.1. Comment gérer l'absence d'usine connectée au bus ?.....	38
9.2. Que faut-il modifier pour gérer plusieurs commandes en parallèle pour un même client ?.....	38
9.3. Que faut-il modifier pour pouvoir gérer plusieurs usines ?.....	39
10. Conclusion.....	40
10.1. Bilan du projet.....	40
10.2. Difficultés rencontrées.....	40
10.3. Ce qui pourrait être amélioré.....	41
10.4. Ce que nous avons appris.....	41

1. Introduction

1.1. Contexte et objectif du projet

Ce projet a été donné dans le cadre de la matière de Java Avancée. Son objectif est de concevoir et d'implémenter un système distribué permettant de commander des lunettes connectées, de les faire fabriquer par une usine automatisée, et de les réceptionner, le tout en passant par un bus de messages asynchrone.

Concrètement, le Readme donnée par notre professeur nous demandait de produire une partie backend et frontend dans deux exécutable distincts qui communiquent entre eux via un broker MQTT :

- La partie Frontend développée en JavaFX, permettant à un utilisateur de parcourir un catalogue de lunettes, de passer commande, de suivre l'avancement de la fabrication en temps réel, et de vérifier la validité d'un numéro de série.
- La partie Backend en Java pur, chargé de recevoir les commandes, de valider leur contenu, de piloter la fabrication via une bibliothèque tierce (fabricateur), et de livrer les lunettes produites au client.

La communication entre les deux composants repose sur le protocole MQTT (Message Queuing Telemetry Transport). Le broker utilisé est Mosquitto, une implémentation open-source du protocole MQTT.

1.2. Présentation de l'équipe et rôles de chacun

Nous avons découpé le projet en plusieurs parties et fonctionnalités afin de pouvoir chacun contribuer à une partie équitable du travail demandé.

Membre	Rôle principal
Riad Belazougui	Lead backend & build
Achraf Raouf	Backend + Client
Inès Jung	Backend + Tests
Flora Bolzon	Client + CI/CD

- Riad Belazougui a joué un rôle de lead sur la partie Backend et s'est occupé du build de l'application. Il a notamment mis en place la structure Maven multi-module, développé l'Usine ainsi que le point d'entrée du backend `App.java`, puis configuré Surefire, le fat JAR et Logback.
- Achraf Raouf a principalement travaillé sur le Backend de l'application, tout en contribuant aussi au Frontend. Il a développé `ServeurCallback.java` et `Serveur.java`. Côté interface, il a réalisé `CommandeController.java` ainsi que la navigation JavaFX.
- Inès Jung s'est concentrée sur le Backend et les tests. Elle a développé `ServeurCallbackTest.java` et a également conçu plusieurs modèles côté client, tels que `CommandeModel`, `LivraisonModel`, `Produit`, etc.
- Flora Bolzon a pris en charge une partie du Frontend ainsi que la CI/CD. Elle a développé l'ensemble des écrans FXML, ainsi que les fichiers `FabricationController.java`, `AccueilController.java` et `App.java`. Elle s'est aussi occupée de la pipeline GitHub Actions et de la configuration jlink multi-plateforme.

La répartition s'est faite naturellement en fonction des compétences de chacun : Riad et Achraf avaient une expérience plus solide en Java backend, tandis que Flora s'est concentrée sur l'interface utilisateur et l'automatisation CI/CD, et Inès a joué un rôle transversal sur les tests et les modèles de données.

1.3. Méthode de travail

Nous avons travaillé de manière itérative et incrémentale :

La collaboration s'est appuyée sur :

- Git et GitHub comme outil central de versioning et de partage du code
- Des sessions de travail en équipe avec synchronisation en début et fin de journée
- Un découpage clair des responsabilités par composant (backend vs. client) pour minimiser les conflits de merge
- Le Test et l'intégration de chaque fonctionnalité avant de passer à la suivante

2. Analyse du cahier des charges et planification

2.1. Lecture et compréhension du sujet

La première étape a été une lecture attentive et collective du cahier des charges (README.md). Nous avons identifié les exigences selon trois catégories :

Exigences fonctionnelles :

- Application JavaFX composée d'au moins 4 écrans : accueil, catalogue/commande, suivi de fabrication, vérification de numéro de série.
- Communication asynchrone entre client et backend via MQTT
- Gestion d'une seule commande en simultané par client (mais plusieurs clients peuvent coexister)
- Validation des commandes avant production (types connus, quantités dans les bornes)
- Livraison des lunettes avec leurs numéros de série
- Vérification de la validité d'un numéro de série

Exigences non-fonctionnelles :

- Application "production-ready" : gestion des erreurs, logs pertinents, paramètres de configuration externalisés.
- Pas de framework, Java pur uniquement.
- Format de sérialisation des messages MQTT à concevoir, JSON interdit.
- L'Usine doit pouvoir traiter plusieurs commandes en parallèle.
- Mutualisation des commandes pour optimiser la productivité.

Exigences de livrable :

- Un JAR prêt à l'emploi pour le client
- Un JAR prêt à l'emploi pour le backend
- Un README d'utilisation
- Ce rapport d'une quarantaine de pages
- Tests unitaires et/ou d'intégration
- CI/CD

2.2. RoadMap initial

Après avoir lu le sujet, nous nous sommes appuyés sur la trame proposée dans le README afin de définir un ordre d'implémentation cohérent. L'objectif était d'avancer progressivement, en commençant par les bases techniques avant de passer à l'intégration complète de l'application.

Nous avons d'abord pris le temps de nous documenter sur MQTT. Il fallait comprendre le fonctionnement du modèle publish/subscribe, installer Mosquitto en local, puis faire quelques tests simples avec `mosquitto_pub` et `mosquitto_sub` pour vérifier l'envoi et la réception de messages.

Ensuite, nous avons étudié la librairie `fabricateur`. Cette étape nous a permis de lire la JavaDoc, d'identifier les contraintes principales, comme la capacité ou l'ordre des appels, puis d'écrire un premier `main` de test pour manipuler la librairie manuellement.

Une fois ces bases posées, nous avons choisi les topics MQTT à utiliser ainsi que le format de sérialisation des messages. Donc les chanel que le Backend et le Frontend vont utiliser pour s'envoyer des messages et la transformation de ces derniers.

Nous sommes ensuite passés à l'implémentation de l'Usine, qui représente le cœur du backend. Cette partie a d'abord été testée avec un `main`, avant d'être intégrée avec MQTT.

L'étape suivante a été le développement du `Serveur MQTT` et du `ServeurCallback`, afin de gérer la connexion au broker, la réception des commandes et leur traitement.

En parallèle, une fois l'API MQTT suffisamment définie, nous avons commencé le développement du client JavaFX, notamment les écrans et la navigation côté interface.

Les tests unitaires ont été ajoutés progressivement, en priorité sur les parties les plus importantes comme l'Usine et le `ServeurCallback`.

Enfin, la CI/CD et le packaging ont été mis en place en dernière partie du projet, lorsque le code était suffisamment stable et fonctionnel.

Dans l'ensemble, ce planning a été plutôt bien respecté, même si quelques ajustements ont été nécessaires en cours de route.

2.3. Recherches préliminaires et sources

Avant de coder, nous avons consacré du temps à de la documentation et de la recherche. Voici les ressources que nous avons consultées, organisées par domaine.

MQTT et Publish/Subscribe

Le paradigme publish/subscribe est le fondement de l'architecture du projet. Nous avons commencé par comprendre les concepts (publishers, subscribers, broker, topics). Nous avons ensuite consulté le site officiel de MQTT, pour comprendre les spécificités du protocole : QoS levels (0, 1, 2), messages retenues, last will testament, et surtout la syntaxe des wildcards de topics.

Eclipse Paho, Client MQTT Java

Pour l'intégration Java, le choix du client MQTT s'est naturellement porté vers la bibliothèque Eclipse Paho, qui est la référence officielle recommandée par mqtt.org. Nous avons consulté sa documentation pour comprendre l'API `MqttClient`, `MqttConnectOptions`, et l'interface `MqttCallback`.

JavaFX 21

Le sujet imposant une application JavaFX, nous avons utilisé la documentation officielle d'OpenJFX comme référence principale, ainsi que la partie 2 du cours de Java avancée. Certains points ont nécessité des recherches spécifiques, tels que la structure FXML et son intégration avec les contrôleurs, le mécanisme `Platform.runLater()` pour les mises à jour UI depuis des threads non-FX, et le problème du déploiement modulaire avec `jlink`.

Maven multi-module

La documentation Apache Maven a été consultée pour la structuration du projet en multi-module avec héritage de POM parent, la gestion des plugins de build.

Ping-pong scheme

Le sujet mentionnait l'utilisation du ping-pong scheme comme modèle de référence pour implémenter le protocole de communication. Nous avons étudié ce schéma pour concevoir le flux de messages entre client et backend (commande → validation → production → livraison).

3. Architecture du système

3.1. Vue d'ensemble

Le système repose sur une architecture événementielle (Event-Driven Architecture) où les composants communiquent exclusivement via des messages publiés sur un broker MQTT. Aucun appel direct (RPC, HTTP, socket) n'existe entre le client et le backend. Ils ne se "connaissent" pas, ils ne font que publier et s'abonner à des topics.

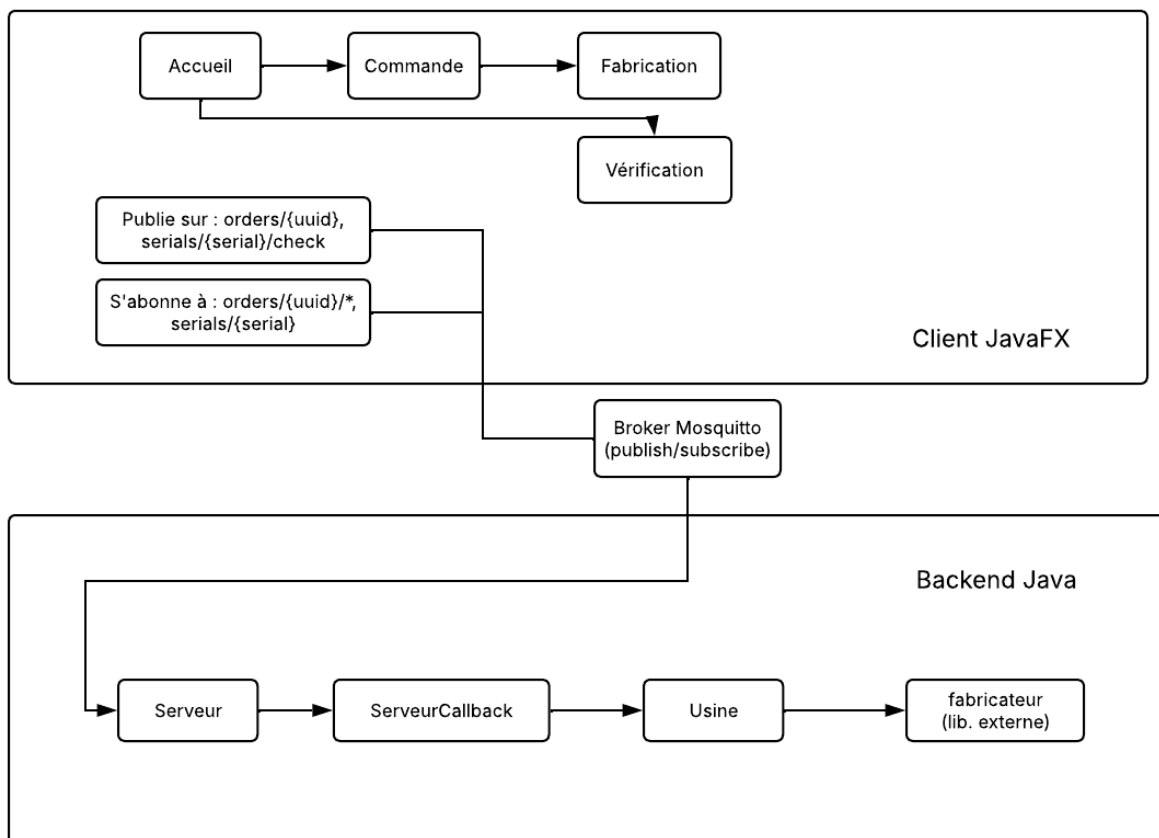


Diagramme de communication entre le Client JavaFx et le Backend Java

Cette architecture présente plusieurs avantages importants pour notre cas d'usage :

- **Indépendance entre les composants** : le client et le backend n'ont pas besoin de connaître leurs adresses IP ou leurs ports respectifs. Ils communiquent uniquement avec le broker MQTT.
- **Souplesse dans les échanges** : un composant peut publier un message même si l'autre n'est pas encore connecté, notamment grâce aux messages conservés (retained messages).

- **Gestion de plusieurs clients** : plusieurs clients peuvent envoyer des commandes en même temps sans nécessiter de modification particulière côté backend.
- **Meilleure tolérance aux interruptions** : si un composant se déconnecte temporairement, cela ne provoque pas forcément la perte des messages, selon le niveau de QoS choisi.

3.2. Choix architecturaux fondamentaux

Pas de Spring (contrainte du sujet)

Le sujet précisait que nous ne devions pas utiliser de framework comme Spring. Nous avons donc travaillé en Java pur, ce qui nous a obligés à organiser nous-mêmes les différentes classes et leurs dépendances.

Concrètement, cela signifie que les objets nécessaires au fonctionnement de l'application sont créés manuellement. Par exemple, dans le backend, `App.java` crée les éléments principaux comme le Fabricateur, l'Usine, le client MQTT et le Serveur, puis les transmet aux classes qui en ont besoin. Cette approche nous a permis de mieux comprendre le rôle de chaque classe et d'éviter d'avoir une architecture trop automatique ou difficile à maîtriser.

Format de sérialisation custom (contrainte du sujet)

Le sujet interdisait aussi l'utilisation d'un format tout fait comme JSON pour les messages MQTT. Nous avons donc choisi de créer un format simple, sous forme de texte.

Pour une commande, le message contient le type de lunette et la quantité souhaitée, par exemple : `CLAUDE:2;BANANA:1`

Pour une livraison, le principe est similaire, mais avec les numéros de série des lunettes fabriquées : `CLAUDE:SN001;BANANA:SN002`

Ce format a l'avantage d'être facile à lire, à écrire et à tester avec les outils MQTT.

Java 17

Nous avons choisi Java 17 comme version pour notre application, car c'est une version LTS, donc stable et adaptée pour un projet de ce type. Elle est aussi compatible avec JavaFX 21, que nous utilisons pour l'application cliente.

Java 17 nous a permis d'utiliser quelques fonctionnalités pratiques, comme les `records` pour représenter certains modèles de données, ou encore `List.of()` pour créer facilement des listes non modifiables. L'objectif n'était pas d'utiliser des fonctionnalités avancées à tout prix, mais plutôt de travailler avec une version récente et propre de Java.

Structure Maven multi-module

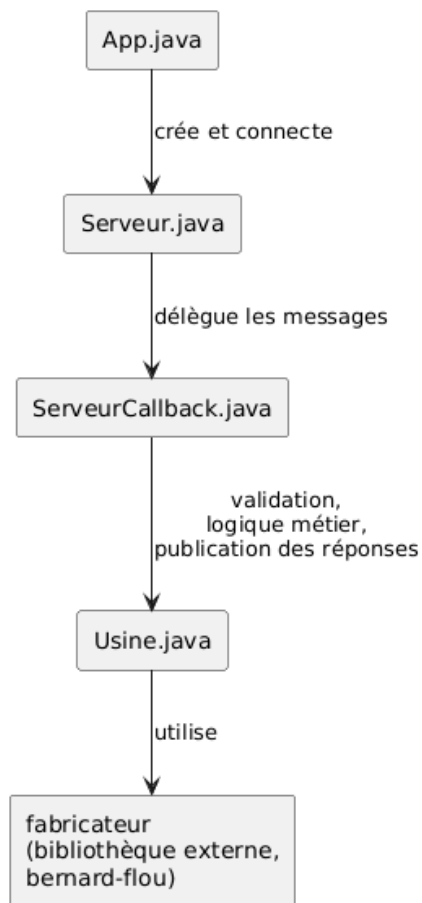
Nous avons organisé le projet avec une structure Maven multi-module. Le projet contient :

- un module parent, qui regroupe la configuration commune ;
- un module backend, qui contient la partie serveur, l'usine et la dépendance vers le fabricant ;
- un module client, qui contient l'application JavaFX et ses propres dépendances.

Cette organisation nous a permis de séparer clairement le backend et le frontend. Chaque partie peut être construite et testée plus facilement, sans mélanger les dépendances. Par exemple, les dépendances JavaFX restent uniquement dans le module client et ne sont pas ajoutées inutilement au backend.

3.3. Architecture backend :

Le backend est découpé en quatre classes avec des responsabilités clairement séparées, suivant le principe de responsabilité unique (SRP) :



App.java : c'est le point d'entrée. Il lit la configuration (URL du broker), instancie tous les composants, démarre le serveur, installe un hook d'arrêt propre, et bloque le thread principal indéfiniment.

Serveur.java : gère uniquement la connexion MQTT et les abonnements. Il crée un `ServeurCallback` et l'enregistre comme handler. Il s'abonne à deux topics : `orders/+` (commandes entrantes) et `serials/+check` (demandes de vérification de numéros de série).

ServeurCallback.java : implémente l'interface `MqttCallback`. C'est le vrai cerveau du backend : il reçoit chaque message entrant, le route vers le bon traitement, valide la commande, appelle l'usine, et publie les réponses.

Usine.java : encapsule toute la logique de production. Elle utilise une `LinkedBlockingQueue` pour recevoir les demandes, un thread daemon dédié pour les traiter, et des `CompletableFuture` pour restituer les résultats aux appelants. La logique de batching (mutualisation) est ici.

3.4. Architecture client JavaFX

Le client JavaFX est organisé selon le principe **MVC** : Modèle, Vue, Contrôleur. Cette organisation permet de séparer l’affichage, la logique de navigation et les données manipulées par l’application.

Les vues correspondent aux fichiers `.fxml`. Ce sont eux qui définissent la structure visuelle des écrans : boutons, textes, champs de saisie, listes, etc.

Les contrôleurs sont des classes Java associées à ces fichiers FXML. Ils gèrent les actions de l’utilisateur, par exemple le clic sur un bouton, la validation d’une commande ou l’affichage d’un résultat.

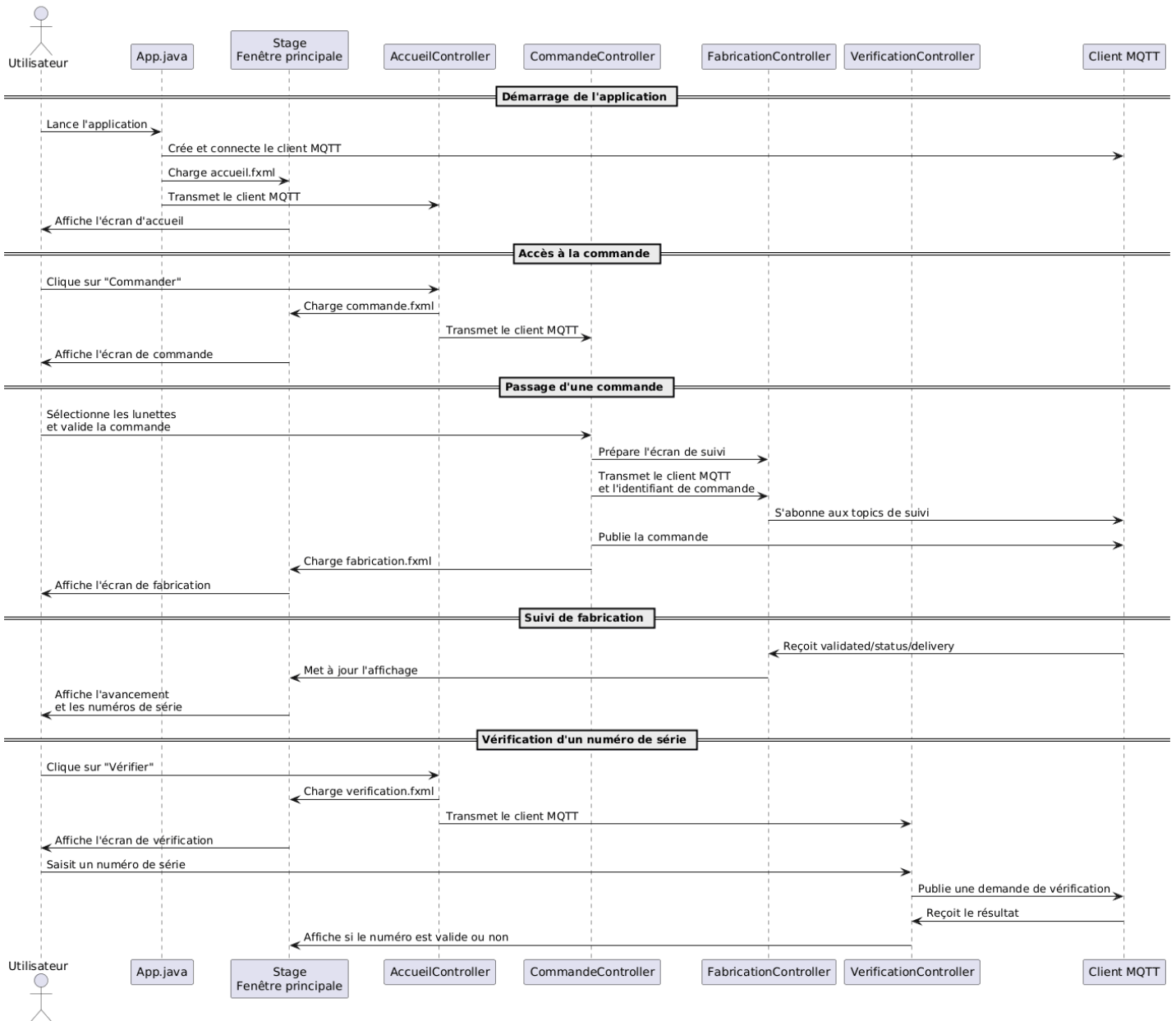
Les modèles sont des classes Java simples qui représentent les données utilisées dans l’application, comme une commande, une livraison ou un produit.

Pour la navigation, nous avons choisi une solution simple : l’application utilise une seule fenêtre principale, appelée `Stage`. À chaque changement d’écran, on remplace simplement la scène affichée dans cette fenêtre. Cela évite d’ouvrir plusieurs fenêtres et rend l’application plus fluide pour l’utilisateur.

Le client MQTT est créé au lancement de l’application, puis transmis aux différents contrôleurs. Cela permet aux écrans qui en ont besoin de publier ou recevoir des messages MQTT.

Le diagramme de séquence de la navigation du client JavaFX se trouve dans la page suivante.

Navigation dans le client JavaFX



3.5. Protocole de communication MQTT

Le protocole de communication est l'élément central du projet. Nous l'avons conçu en nous basant sur le "ping-pong scheme" décrit dans le sujet, et en suivant le tableau de topics imposé par le cahier des charges.

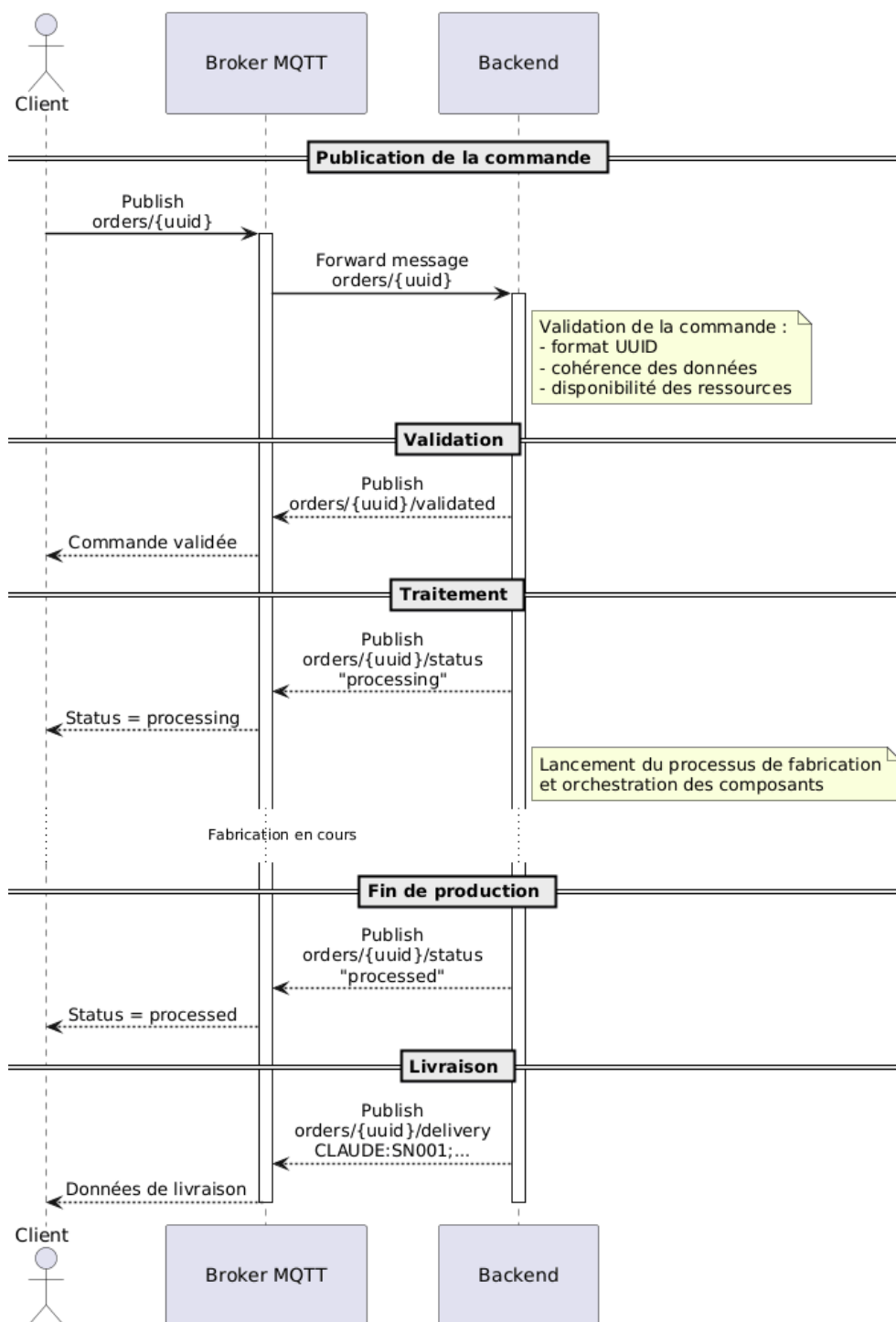
Topics et sens des messages :

Topic	Sens	Données	Description
orders/{uuid}	Client → Backend	Commande sérialisée	Passer une commande
orders/{uuid}/validated	Backend → Client	(vide)	Commande valide, production démarrée
orders/{uuid}/cancelled	Backend → Client	Message d'erreur	Commande invalide, transaction terminée
orders/{uuid}/status	Backend → Client	processing ou processed	Mise à jour de progression
orders/{uuid}/delivery	Backend → Client	Livraison sérialisée	Commande terminée, numéros de série
orders/{uuid}/error	Backend → Client	Message d'erreur	Erreur de production, transaction terminée
serials/{serial}/check	Client → Backend	(vide)	Demande de vérification d'un numéro de série
serials/{serial}	Backend → Client	Type de lunette ou invalid	Résultat de la vérification

L'identifiant {uuid} est un UUID v4 généré par le client au moment de la commande, ce qui garantit l'unicité globale conformément aux exigences du sujet.

Le diagramme de séquence montrant le cycle de traitement d'une commande est disponible à la page suivante.

Cycle complet de traitement d'une commande MQTT



4. Les dépendances : comment et pourquoi

Cette section documente chaque dépendance ajoutée au projet : pourquoi nous en avons besoin, comment nous l'avons trouvée, et comment elle est utilisée. C'est une exigence explicite du cahier des charges.

Structure Maven multi-module :

Le fichier `pom.xml` à la racine du projet est un POM parent qui déclare deux sous-modules (backend et client) et centralise les dépendances communes aux deux. Les `pom.xml` des modules héritent du parent.

Pourquoi ce choix : Cette structure permet de ne déclarer qu'une seule fois les versions de dépendances communes, afin d'éviter les désynchronisations de versions entre modules, et de builder tout le projet avec une seule commande à la racine.

Eclipse Paho MQTT Client :

Pourquoi cette dépendance : Le sujet impose l'utilisation du protocole MQTT. Il faut donc une bibliothèque Java capable d'agir comme client MQTT, créer des connexions, publier des messages, s'abonner à des topics, et recevoir des messages de manière asynchrone via un callback.

Eclipse Paho est la bibliothèque cliente MQTT officielle maintenue par la Fondation Eclipse, directement référencée par le site officiel mqtt.org comme implémentation de référence pour Java. Elle est disponible sur Maven Central. La version 1.2.5 est la dernière version stable.

Le composant `MqttClient` est utilisé pour établir la connexion et échanger des messages, aussi bien du côté du serveur dans `Serveur.java` que du côté du client JavaFX dans `App.java`. `MqttConnectOptions` permet de configurer les paramètres de connexion, par exemple la reconnexion automatique ou les délais de connexion.

L'interface `MqttCallback`, implémentée dans `ServeurCallback.java`, sert à recevoir automatiquement les messages envoyés.

Enfin, `MqttMessage` représente le contenu des messages MQTT envoyés ou reçus.

SLF4J API

Cette dépendance a été choisie car le sujet demande une application « production-ready » avec des journaux d'exécution pertinents. Utiliser simplement les `sysout` n'est pas adapté dans un projet Java professionnel, car cela ne permet ni de gérer différents niveaux de logs, ni de personnaliser l'affichage ou la destination des messages. SLF4J fournit une solution plus propre et plus flexible pour gérer les logs dans l'application.

Cette bibliothèque a également été retenue car SLF4J est aujourd'hui un standard très répandu dans le développement Java. Elle est largement utilisée dans les projets modernes.

Logback Classic

Cette dépendance a été choisie car SLF4J ne permet pas à lui seul d'écrire les logs : il a besoin d'une implémentation concrète. Logback a donc été utilisé pour gérer l'affichage et la configuration des journaux d'exécution. Cette bibliothèque permet notamment de personnaliser le format des logs, de choisir leur destination (console, fichier, etc.) et de définir différents niveaux de logs selon les parties de l'application grâce au fichier logback.

Logback a été retenu car il s'agit de l'implémentation de référence de SLF4J et d'une solution très répandue dans les projets Java modernes.

Un problème a toutefois été rencontré lors de la création du client en « fat JAR » avec le plugin Shade. Après l'empaquetage, les logs ne fonctionnaient plus correctement. Le problème venait du service `Loader` utilisé par Logback pour être détecté automatiquement par SLF4J. Lors de la fusion des différents JARs, certains fichiers internes étaient écrasés. Pour résoudre ce problème, un `Service Resource Transformer` a été ajouté à la configuration du plugin Shade afin de fusionner correctement ces fichiers au lieu de les remplacer.

JUnit 5 Jupiter

Cette dépendance a été choisie car le sujet met en avant l'importance des tests unitaires. JUnit 5 est aujourd'hui le framework de référence pour réaliser des tests en Java. Il permet de vérifier le bon fonctionnement des différentes parties de l'application de manière automatisée.

La version 5 apporte également plusieurs améliorations par rapport aux anciennes versions, notamment des annotations plus claires et plus pratiques, une organisation des tests plus flexible et une meilleure compatibilité avec d'autres outils comme Mockito.

JUnit 5 a été retenu car il s'agit du standard le plus utilisé dans les projets Java modernes. Toute l'équipe connaissait déjà cette bibliothèque, ce qui a facilité sa mise en place dans le projet.

Mockito

Cette dépendance a été choisie car certaines classes du projet possèdent des dépendances externes difficiles à utiliser dans de vrais tests unitaires. Par exemple, certaines fonctionnalités reposent sur un fabricant, une bibliothèque propriétaire, ou sur `MQTTClient`, qui nécessite normalement un broker MQTT actif. Mockito permet de remplacer ces dépendances par de faux objets appelés « mocks » afin de tester uniquement la logique de l'application.

Grâce à Mockito, il est possible de simuler des comportements précis, de définir les valeurs retournées par certaines méthodes et de vérifier que certaines actions ont bien été exécutées pendant les tests. Cela permet de réaliser des tests unitaires plus simples, plus rapides et indépendants des composants externes.

Un problème a toutefois été découvert au début du développement des tests. JUnit 5 avait bien été ajouté au projet, mais Mockito avait été oublié dans le fichier de configuration Maven. L'erreur a été repérée au moment de l'écriture des premiers véritables tests unitaires et la dépendance manquante a ensuite été ajoutée.

Maven Surefire Plugin

Cette dépendance a été ajoutée car la version du plugin Surefire fournie par défaut avec Maven est trop ancienne pour prendre correctement en charge JUnit 5. Sans configuration explicite, aucun test ne se lançait, tout en indiquant malgré tout que l'exécution s'était bien déroulée, ce qui rendait le problème difficile à détecter.

Le problème a été découvert lors des premiers lancements de l'intégration continue, lorsque les tests n'étaient jamais exécutés. La cause a rapidement été identifiée : il fallait déclarer explicitement une version plus récente du plugin Surefire, compatible avec JUnit 5, dans le fichier pom.xml parent.

Bibliothèque fabricant

Cette dépendance a été utilisée car elle était imposée par le sujet du projet. Il s'agit d'une bibliothèque propriétaire simulant une machine de fabrication de lunettes. Elle fournit une API permettant de configurer la machine et de produire les lunettes demandées par l'application.

JavaFX 21

JavaFX est un framework Java utilisé pour créer des interfaces graphiques modernes. Il permet de développer des applications avec des fenêtres, des boutons, des formulaires et d'autres éléments visuels. JavaFX facilite aussi l'organisation du code en séparant l'interface graphique de la logique métier, notamment grâce à FXML qui permet de définir l'interface dans des fichiers dédiés.

Pour l'utiliser, il faut ajouter les dépendances JavaFX dans le projet Maven et utiliser la bonne version compatible avec Java (ici JavaFX 21). Il faut également préciser les modules nécessaires dans `module-info.java` pour que l'application reconnaisse les composants graphiques. Enfin, comme JavaFX dépend de bibliothèques natives propres à chaque système d'exploitation, il est important de bien inclure les bons fichiers selon la plateforme utilisée pour que l'application fonctionne correctement.

Gson

Gson est une bibliothèque développée par Google qui permet de lire et de convertir des fichiers JSON en objets Java. Dans le projet, elle est utilisée pour charger le fichier `products.json`, qui contient le catalogue de produits du client, et le transformer en objets exploitables par l'application.

Son utilisation est très simple : il suffit de lire le fichier JSON pour le convertir en tableau ou en liste d'objets Java. Cela permet d'éviter de parser manuellement le JSON et rend le code plus clair et plus maintenable. Elle a été choisie car elle est légère, très répandue et disponible sur Maven Central.

Maven-shade-plugin

Maven Shade permet de créer un fichier JAR exécutable contenant à la fois le code de l'application et toutes ses dépendances. Dans le projet, il est utilisé pour le backend afin de produire un seul fichier exécutable pouvant être lancé directement avec `java -jar`, sans avoir à installer ou gérer les bibliothèques externes séparément.

Ce plugin a été choisi car c'est la solution standard dans l'écosystème Maven pour générer ce type de "fat JAR". Il a été ajouté après avoir constaté que le JAR généré par défaut ne contenait pas les dépendances nécessaires au fonctionnement de l'application. En revanche, il n'a pas pu être utilisé pour le client JavaFX, car les bibliothèques natives de JavaFX ne sont pas compatibles avec ce type d'assemblage, ce qui a conduit à utiliser jlink pour cette partie du projet.

Javafx-maven-plugin et jlink

jlink, est un outil intégré à Java permettant de créer une version légère de Java contenant uniquement les modules nécessaires à l'application. Dans le projet, il est utilisé pour distribuer le client JavaFX sans obliger l'utilisateur à installer Java manuellement. L'application et l'environnement Java nécessaire sont directement fournis ensemble dans une archive prête à être utilisée.

Cette solution a été trouvée dans la documentation officielle d'OpenJFX lors des recherches sur le déploiement d'applications JavaFX sur plusieurs systèmes d'exploitation. Grâce à jlink, le projet génère automatiquement des archives distinctes pour Windows, Linux et macOS, chacune contenant l'application ainsi qu'un script de lancement permettant de démarrer facilement le client.

5. Implémentation détaillée

5.1. L'Usine / cœur du backend

L'Usine est la classe la plus complexe du projet. Elle encapsule toute la logique de production et gère la concurrence entre les appels simultanés.

Version initiale, production synchrone simple

La première implémentation de `produire()` était synchrone et simple : convertir la map de commande en liste de types, découper en lots selon la capacité du fabricant, configurer et fabriquer chaque lot.

Logique de découpage en lots

La contrainte du fabricant est que `configurer()` accepte un tableau dont la taille ne peut pas dépasser `fabricateur.getCapacity()`. Si une commande contient plus de lunettes que la capacité, on doit fabriquer en plusieurs lots :

Exemple : capacité = 4, commande = {CLAUDE : 3, BANANA : 3} = 6 lunettes

Lot 1 : [CLAUDE, CLAUDE, CLAUDE, BANANA] → configurer + fabriquer ×4

Lot 2 : [BANANA, BANANA] → configurer + fabriquer ×2

Version finale, threading et concurrence

Le sujet exigeait que `produire()` puisse être appelé par plusieurs threads simultanément (plusieurs commandes en cours en même temps). La version synchrone ne le permettait pas car `fabricateur.configurer()` n'est pas thread-safe.

La solution implémentée repose sur le pattern producteur-consommateur avec `LinkedBlockingQueue`, chaque appel à `produire()` crée une `Demande` (contenant la liste de types et un `CompletableFuture`) et la dépose dans la file. Un thread daemon unique (`boucleProduction()`) consomme la file en continu, il extrait d'abord la première demande (bloquant), puis draine toutes les demandes qui sont déjà en attente dans la file, c'est la mutualisation. Il traite le groupe de demandes collectées, en répartissant les lots entre elles. Chaque demande reçoit ses résultats via `CompletableFuture.complete()`.

Exemple de mutualisation :

3 commandes arrivent "simultanément", capacité fabricant = 8 :

Commande A : 2 CLAUDE

Commande B : 2 BANANA

Commande C : 1 LE_CHAT

Batch groupe : [CLAUDE, CLAUDE, BANANA, BANANA, LE_CHAT] = 5 lunettes → 1 seul lot

Sans mutualisation : 3 appels séparés à `configurer()` + `fabriquer()`

Avec mutualisation : 1 seul appel à `configurer()` + 5 appels à `fabriquer()`

5.2. Le Serveur et le callback MQTT

La classe `Serveur` est intentionnellement simple, son rôle est de gérer la connexion MQTT et celle des abonnements.

Bug `orders/#` et `orders/+`

L'un des premiers bugs est apparu lors de la première version du serveur qui utilisait `orders/#` au lieu de `orders/+`. Le `#` correspond à tous les sous-niveaux de `orders/`, donc `orders/abc/validated` ou `orders/abc/delivery` étaient reçus par le backend.

Résultat : quand le backend publiait `orders/{id}/validated`, il recevait immédiatement son propre message de confirmation. Croyant recevoir une nouvelle commande, il la traitait à nouveau, réenvoyant une confirmation, et tournait ainsi en boucle infinie. Ce bug a été découvert lors des premiers tests d'intégration manuels.

La correction a consisté à remplacer `#` par `+`, qui ne correspond qu'à exactement un niveau de dossier. Le symbole `+` est plus strict : il ne correspond qu'à un seul niveau de dossier. Désormais, le serveur ignore les sous-niveaux comme `/validated`. Il ne reçoit plus ses propres confirmations, ce qui a définitivement résolu la boucle infinie.

`ServeurCallback.java`

Le `ServeurCallback` reçoit tous les messages et les route selon le préfixe du topic :

Pour les commandes, le traitement suit ces étapes :

1. Filtrage des messages : Ignorer les notifications qui dépassent 2 segments de topic (ex. : ne traiter que `orders/{uuid}` et rejeter `orders/{uuid}/validated`).
2. Désérialisation : Convertir la chaîne de caractères brute (ex. : `"CLAUDE:2;BANANA:1"`) en un dictionnaire structuré `Map<TypeLunette, Integer>`.
3. Validation des données : Vérifier la conformité de la commande :
 - Les modèles de lunettes sont-ils connus ?
 - Les quantités par modèle sont-elles comprises entre 0 et 9 ?
 - Le total d'articles est-il strictement supérieur à 0 ?
4. Gestion des échecs de validation : Si une règle échoue, publier l'état d'annulation sur `orders/{uuid}/cancelled` et stopper immédiatement le processus.
5. Confirmation : Publier la validation de la commande sur `orders/{uuid}/validated`.
6. Lancement : Mettre à jour le statut en publiant `"processing"` sur `orders/{uuid}/status`.
7. Production (Bloquant) : Déclencher l'appel synchrone à l'usine : `usine.produire(commande)`.
8. Fin de fabrication : Mettre à jour le statut en publiant `"processed"` sur `orders/{uuid}/status`.

9. Expédition : Sériialiser les informations de livraison et les publier sur `orders/{uuid}/delivery`.
10. Gestion des anomalies : En cas d'erreur ou d'exception critique à n'importe quelle étape du cycle, intercepter le problème et publier une alerte sur `orders/{uuid}/error.</TypeLunette,>`

5.3. Le client JavaFX

Le point d'entrée du client gère la connexion MQTT et le chargement de l'écran d'accueil. Il installe un gestionnaire de fermeture de fenêtre pour déconnecter proprement le client MQTT.

Le contrôleur de commande charge les 4 produits depuis `products.json`, les affiche dans des cartes UI avec des menus déroulants pour les quantités, puis envoie la commande MQTT et navigue vers l'écran de suivi.

Un point de conception important : la navigation vers `FabricationController` se fait avant la publication MQTT. Cela garantit que le contrôleur de fabrication est prêt à recevoir les messages de réponse avant même que la commande ne soit envoyée, évitant ainsi une condition de course où `validated` arriverait avant que le contrôleur ne soit abonné.

C'est le contrôleur le plus complexe du client. Il s'abonne à 5 topics, affiche les mises à jour en temps réel, gère un timeout de 30 secondes, et nettoie ses abonnements à la fin.

Les callbacks MQTT s'exécutent dans des threads non-FX. Toute modification de l'interface utilisateur doit être faite dans le thread JavaFX via `Platform.runLater()` :

Un système de timeout de 30 secondes a été mis en place afin d'éviter qu'une commande reste bloquée indéfiniment lorsqu'aucune usine ne répond. Lorsqu'une commande est envoyée, une tâche planifiée démarre un compte à rebours de 30 secondes. Si aucune réponse n'est reçue avant la fin du délai, l'interface affiche automatiquement un message indiquant qu'aucune usine n'est disponible, puis la procédure se termine proprement.

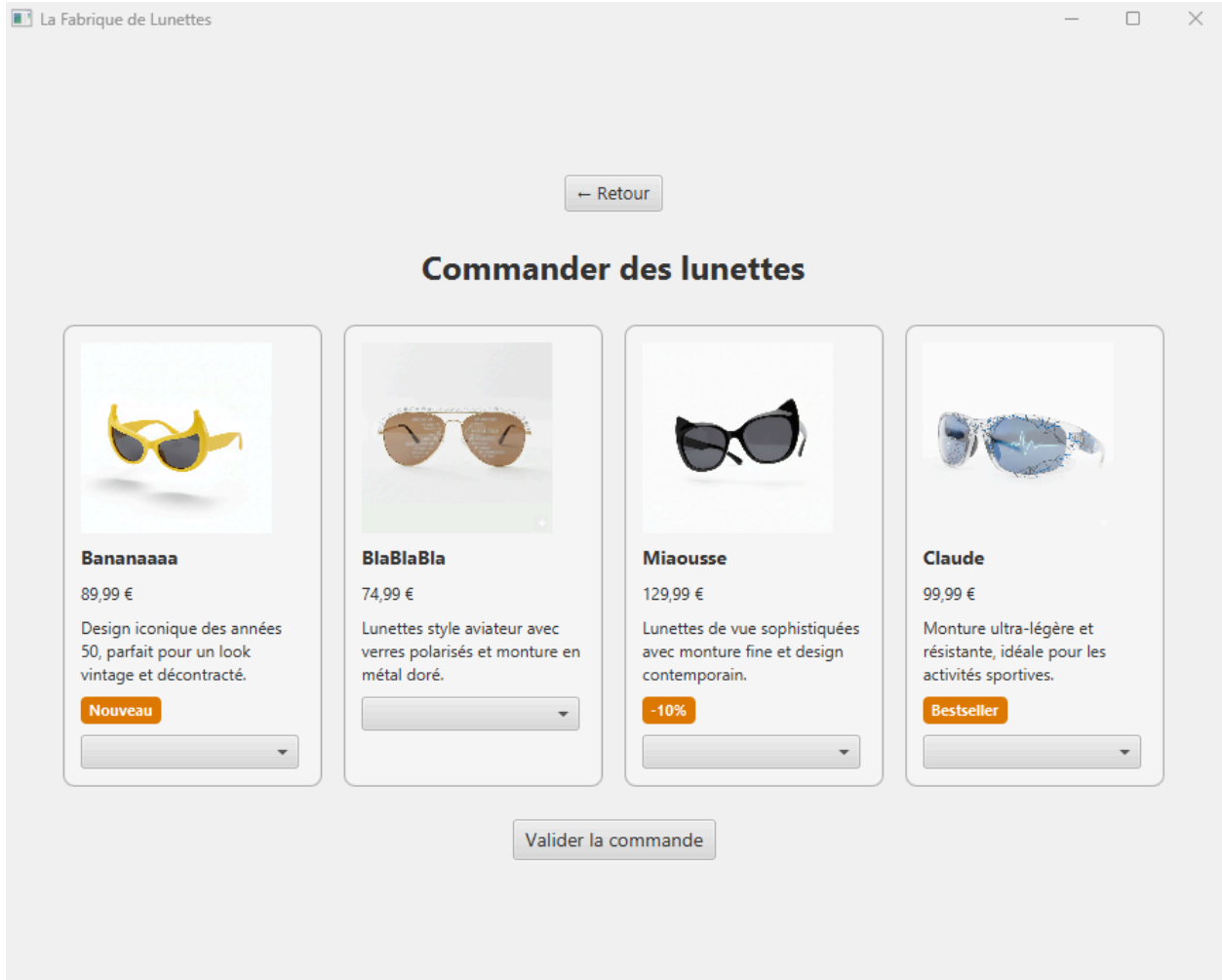
Dans `VerificationController.java`, une gestion particulière des abonnements MQTT a également été ajoutée. Comme l'utilisateur peut vérifier plusieurs numéros de série à la suite, il était important d'éviter d'accumuler des abonnements inutiles aux anciens topics MQTT. Avant chaque nouvelle vérification, l'application se désabonne donc du précédent topic de réponse, puis s'abonne au nouveau correspondant au numéro de série demandé. Cela garantit que seuls les messages liés à la vérification en cours sont reçus et affichés.

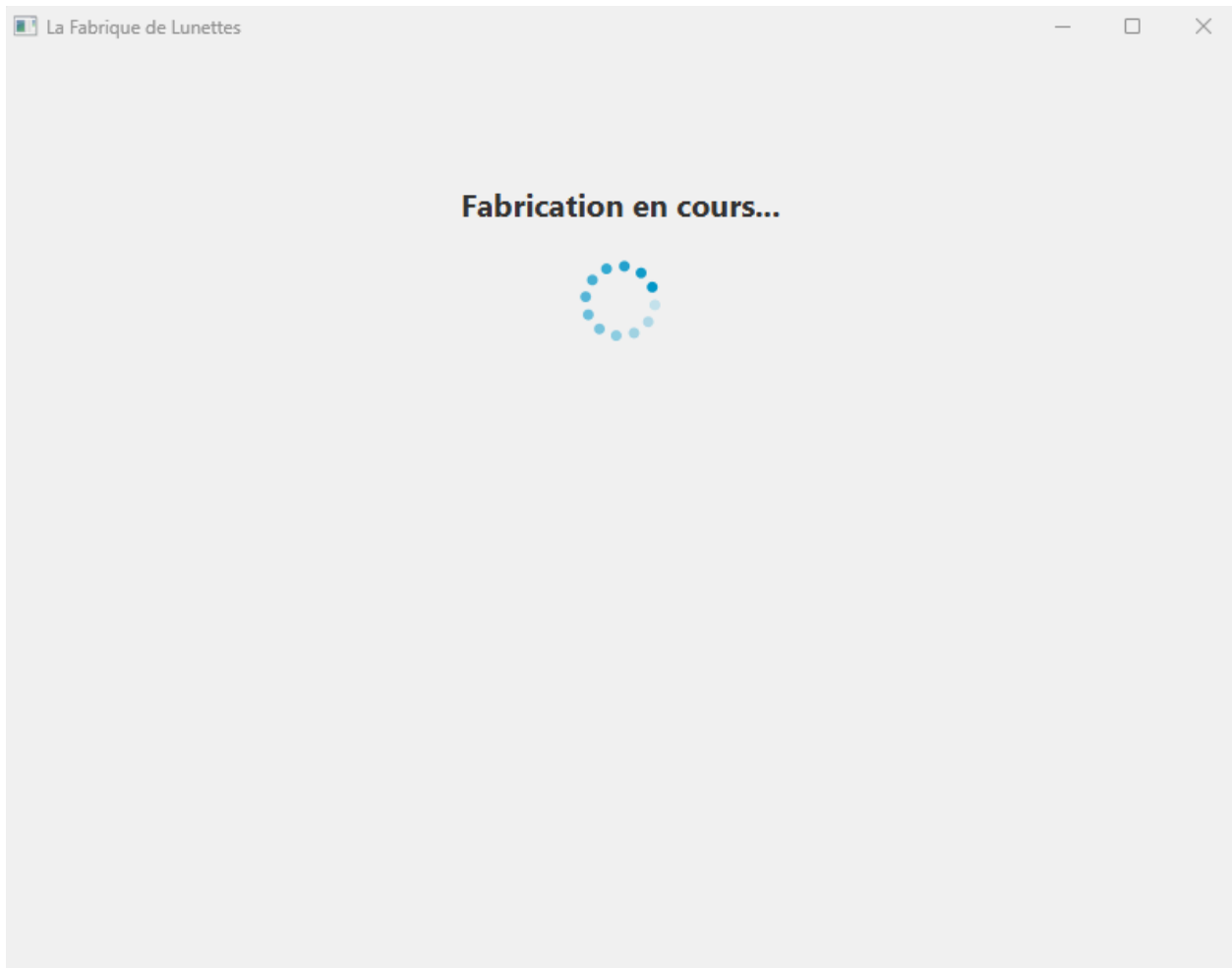
La Fabrique de Lunettes

La nouvelle génération de lunettes connectées

Commander

Vérifier un numéro de série





← Retour

Commande livrée ! (4 lunettes)

Claude (1)

- CL-8RZNBQ-ECFB7991

BlaBlaBla (1)

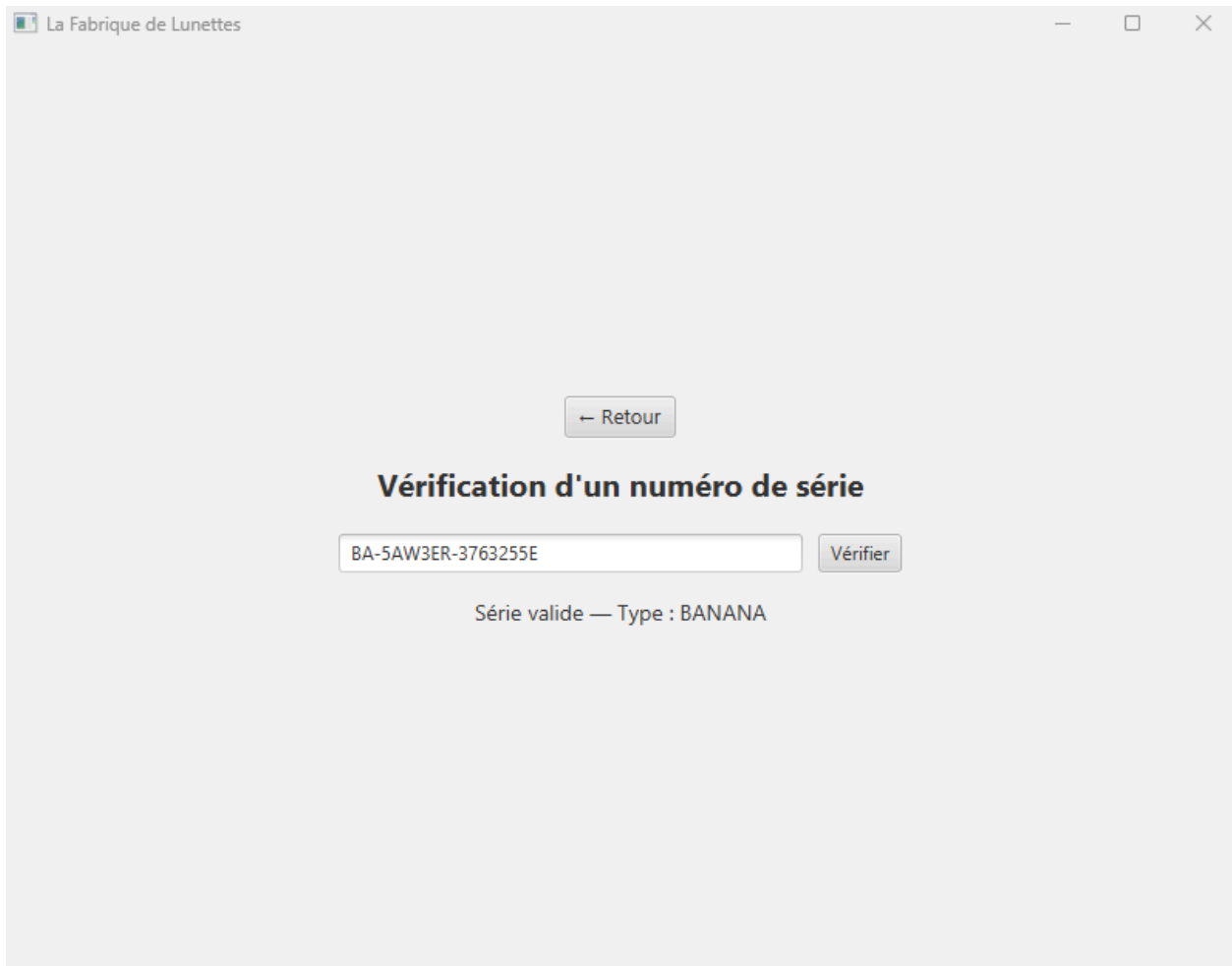
- CH-8EPRG-C1A584B4

Miaousse (1)

- LE-9QBR79-5F5D9533

Bananaaaa (1)

- BA-5AW3ER-3763255E



5.4. Le protocole de sérialisation custom

Le sujet interdisait l'utilisation du format JSON pour les messages MQTT. Nous avons donc créé un format textuel simple basé sur le principe des query strings HTTP. Les commandes envoyées par le client vers le backend utilisent le format `TYPE : quantite`, avec plusieurs éléments séparés par des points-virgules. Par exemple, une commande pouvait être représentée sous la forme `CLAUDE : 2 ; BANANA : 1`. Les réponses du backend utilisent le même principe, mais avec les numéros de série des lunettes fabriquées à la place des quantités.

Ce format a été choisi car il est simple à lire et à manipuler. Le découpage des données peut être réalisé facilement avec des `split(";")` puis `split(":")`, ce qui rend le parsing rapide à implémenter. Il est également pratique pour le débogage, car les messages restent lisibles directement dans des outils comme `mosquitto_sub`. Enfin, ce format est léger et facilement extensible : il suffit d'ajouter un nouveau type de lunette sans modifier la structure générale des messages.

5.5. Gestion des erreurs

La gestion des erreurs faisait partie des exigences du cahier des charges et a été prise en compte à plusieurs niveaux de l'application. Côté backend, chaque commande est vérifiée avant d'être envoyée à l'usine. Si une commande est invalide, son traitement est immédiatement arrêté et un message `cancelled` est envoyé afin d'informer le client qu'elle ne peut pas être exécutée.

Lors de la production, si une erreur survient dans l'usine, par exemple une panne ou une exception inattendue, le backend publie un message `error` sur le topic MQTT correspondant à la commande. Le client peut alors afficher une erreur claire à l'utilisateur. De son côté, le client JavaFX gère également les cas où aucune usine ne répond : après 30 secondes sans réponse, un message de `timeout` est affiché et les ressources sont libérées proprement.

L'application gère aussi les problèmes de connexion au broker MQTT. Si la connexion échoue au démarrage du client, une boîte de dialogue d'erreur s'affiche puis l'application se ferme correctement afin d'éviter un état incohérent. Enfin, les clients MQTT du backend et du client JavaFX utilisent la reconnexion automatique, ce qui permet de rétablir la connexion automatiquement après une coupure temporaire du broker.

6. Tests

6.1. Stratégie de test

Nous avons opté pour des tests unitaires purs, sans intégration avec un vrai broker MQTT ou la vraie bibliothèque fabricant. Cette décision est motivée par :

La rapidité d'exécution (pas de démarrage de services externes)

La reproductibilité (les tests ne dépendent pas d'un état externe)

Le contrôle : on peut tester des comportements difficiles à reproduire avec une vraie machine (erreur de fabrication, commande invalide)

Les dépendances externes sont mockées avec Mockito : `Fabricateur`, `MqttClient`, `Usine`.

Les tests couvrent les couches qui contiennent de la logique métier : `Usine`, `ServeurCallback`, `CommandeModel`, `LivraisonModel`. Certaines classes n'ont pas besoin d'être testées (`Serveur`, `App`, les contrôleurs JavaFX). Tester des contrôleurs JavaFX nécessiterait TestFX et était hors périmètre.

6.2. Tests backend / UsineTest :

	Nom du test	Scénario	Résultat attendu
1	<code>produire_commandePlusPetiteQueCapacite_unSeulLot</code>	2 CLAUDE, capacité 5	2 lunettes retournées, 1 appel à <code>configurer()</code>
2	<code>produire_commandeVide_retourneListeVide</code>	<code>Map.of()</code>	Liste vide, 0 appel à <code>configurer()</code>
3	<code>produire_commandeEgaleCapacite_unSeulLot</code>	5 lunettes, capacité 5	5 lunettes, 1 appel à <code>configurer()</code>
4	<code>produire_commandeSuperieure_deuxLots</code>	6 lunettes, capacité 4	6 lunettes, 2 appels à <code>configurer()</code>
5	<code>produire_multiTypes_tousRetournes</code>	{CLAUDE:2, BANANA:2}, capacité 5	4 lunettes (2 de chaque)
6	<code>produire_demandesParalleles_resultatsCorrects</code>	3 threads simultanés	Chaque thread reçoit ses lunettes

Le test 6 (parallélisme) est particulièrement important : il lance 3 threads appelant `produire()` simultanément et vérifie que chaque thread reçoit bien ses propres lunettes, pas celles des autres.

6.3. Tests backend / ServeurCallbackTest

Tests de `deserialiserCommande()` :

	Scénario	Résultat attendu
1	"CLAUDE:2"	Map {CLAUDE → 2}
2	"CLAUDE:2;BANANA:3"	Map {CLAUDE → 2, BANANA → 3}
3	"INCONNU:2"	null (type inconnu)
4	"CLAUDE" (sans :)	null (format invalide)
5	"CLAUDE:abc"	null (quantité non numérique)
6	"" (vide)	null

Tests de `validerCommande()` :

	Scénario	Résultat attendu
7	{CLAUDE:2, BANANA:3}	null (pas d'erreur)
8	{CLAUDE:0}	Message d'erreur (total nul)
9	{CLAUDE:10}	Message d'erreur (hors limite)
10	{CLAUDE:9}	null (limite valide)
11	{CLAUDE:-1}	Message d'erreur (négatif)
12	Map.of()	Message d'erreur (map vide)

Tests de `messageArrived()` — flux complet :

	Scénario	Vérification
13	Commande valide "CLAUDE:1"	validated + delivery publiés
14	Format invalide "FORMAT_INVALIDE"	cancelled publié, pas de validated
15	Quantité hors limite "CLAUDE:10"	cancelled publié
16	Quantité totale nulle "CLAUDE:0"	cancelled publié
17	Usine lève une exception	validated + error publiés, pas de delivery
18	Topic à 3 segments "orders/uuid/validated"	Aucune publication (topic ignoré)
19	Commande valide avec status	status/processing + status/processed publiés

6.4. Tests client / CommandeModelTest et LivraisonModelTest

CommandeModelTest.java :

	Scénario	Résultat attendu
1	Ajouter ("claude", 2) → serialiser()	"CLAUDE:2"
2	Ajouter deux produits → serialiser()	"CLAUDE:2;BANANA:1"
3	Ajouter avec quantité 0	Exclu de la sérialisation
4	estVide() sur commande vide	true
5	getLignes() → modifier la map retournée	UnsupportedOperationException (immuable)

LivraisonModelTest.java :

	Scénario	Résultat attendu
1	Désérialiser "CLAUDE:SN1"	1 lunette, type CLAUDE
2	Désérialiser "CLAUDE:SN1;BANANA:SN2"	2 lunettes
3	Segment malformé ignoré	Pas d'exception
4	grouperParType()	Map {CLAUDE: [SN1, SN2], BANANA: [SN3]}
5	getNombreLunettes()	Compte correct

6.5. Exécution et intégration CI

Les tests s'exécutent avec :

`mvn test`

Le plugin Surefire 3.2.5 détecte automatiquement les classes de test JUnit 5 (toutes les classes finissant par Test). Les rapports XML sont générés dans `target/surefire-reports/`.

En CI (GitHub Actions), le job tests exécute `mvn -B test --no-transfer-progress` et publie les résultats via l'action `dorny/test-reporter` au format JUnit, ce qui affiche les résultats directement dans l'interface GitHub.

GitHub Actions / Resultats JUnit

succeeded 24 minutes ago in 1s

47 passed, 0 failed and 0 skipped

tests 47 passed

Report	Passed	Failed	Skipped	Time
backend/target/surefire-reports/TEST-com.lunettes.ServeurCallbackTest.xml	20 ✓			208ms
backend/target/surefire-reports/TEST-com.lunettes.UsineTest.xml	6 ✓			1s
client/target/surefire-reports/TEST-com.lunettes.model.CommandeModelTest.xml	11 ✓			21ms
client/target/surefire-reports/TEST-com.lunettes.model.LivraisonModelTest.xml	10 ✓			215ms

✓ backend/target/surefire-reports/TEST-com.lunettes.ServeurCallbackTest.xml

20 tests were completed in 208ms with 20 passed, 0 failed and 0 skipped.

Test suite	Passed	Failed	Skipped	Time
com.lunettes.ServeurCallbackTest	20 ✓			208ms

✓ com.lunettes.ServeurCallbackTest

✓ messageArrived_commandeValide_publieValidatedEtDelivery

✓ validerCommande_quantiteNeuf_pasErreur

✓ messageArrived_serialSansCheck_ignore

✓ messageArrived_commandeInvalide_publieCancelled

✓ deserialiserCommande_casNominal_plusieursTypes

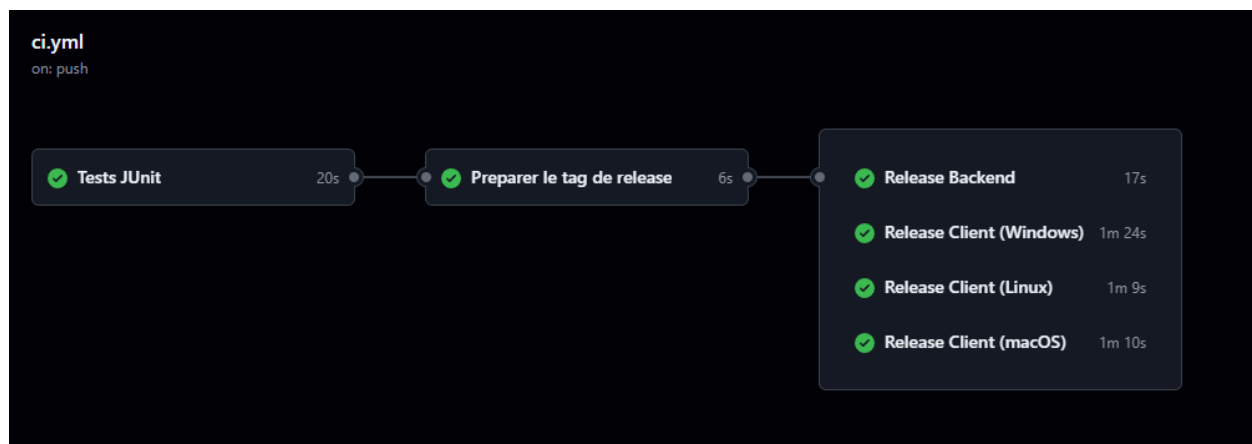
Nous avons choisi GitHub Actions pour gérer l'intégration et le déploiement continu du projet. Cette solution s'intègre directement avec GitHub et permet d'automatiser les tests, la compilation et la génération des livrables sans utiliser d'outil externe supplémentaire. Elle propose également des environnements Windows, Linux et macOS, ce qui était indispensable pour produire les versions du client JavaFX pour chaque système d'exploitation. La configuration du pipeline est stockée dans le fichier `.github/workflows/ci.yml`, ce qui permet de versionner la configuration avec le reste du projet.

Lorsque les tests réussissent sur la branche principale, un pipeline de release démarre automatiquement. Il génère un numéro de version unique basé sur la date et le numéro du build

GitHub Actions, puis crée une release GitHub. Ensuite, plusieurs jobs s'exécutent en parallèle pour produire les différents livrables du projet : le backend sous forme de fat JAR et les clients JavaFX pour Windows, Linux et macOS.

Pour le client JavaFX, le pipeline utilise jpackage afin de créer des applications natives prêtes à être utilisées. Chaque livrable contient directement son propre environnement Java grâce à jlink, ce qui signifie que l'utilisateur n'a pas besoin d'installer Java sur sa machine. Sous Windows et Linux, les applications sont distribuées sous forme d'archives .zip, tandis que macOS utilise un fichier .dmg installable.

Le backend est distribué différemment. Comme il s'agit d'une application en ligne de commande sans interface graphique, il est fourni sous forme d'un fat JAR généré avec Maven Shade. Ce fichier contient déjà toutes les dépendances nécessaires et peut être lancé simplement avec la commande `java -jar backend-runnable.jar`.



8. Conformité au cahier des charges

Cette section présente une vérification systématique de chaque exigence du cahier des charges.

8.1. Exigences fonctionnelles

Exigence	Statut	Où dans le code
Application JavaFX avec au moins 4 écrans	Implémenté	accueil.fxml, commande.fxml, fabrication.fxml, verification.fxml
Écran d'accueil présentant l'application	Implémenté	AccueilController.java
Écran catalogue + commande de plusieurs paires	Implémenté	CommandeController.java – 4 produits, ComboBox 0-9
Écran de suivi de fabrication + numéros de série	Implémenté	FabricationController.java – abonnements MQTT, affichage résultats
Écran de vérification numéro de série	Implémenté	VerificationController.java
Communication via MQTT	Implémenté	Eclipse Paho, broker Mosquitto
Une seule commande simultanée par client	Implémenté	Architecture mono-commande (un UUID actif par client)
Plusieurs clients en parallèle	Supporté	Usine avec LinkedBlockingQueue, ordres traités séquentiellement mais empilables
Commande identifiée par un UUID unique	Implémenté	UUID.randomUUID() dans CommandeController

Exigence	Statut	Où dans le code
Validation des commandes (types connus)	Implémenté	<code>ServeurCallback.deserialiserCommande()</code> + <code>TypeLunette.valueOf()</code>
Validation quantité [0, 10[	Implémenté	<code>ServeurCallback.validerCommande()</code>
Validation total > 0	Implémenté	<code>ServeurCallback.validerCommande()</code>
Commande invalide → topic cancelled	Implémenté	<code>ServeurCallback.traiterCommande()</code>
Livraison avec numéros de série	Implémenté	Topic <code>orders/{uuid}/delivery</code>
Topic validated	Implémenté	Publié avant la production
Topic delivery	Implémenté	Publié après la production
Topic error	Implémenté	Publié si exception pendant la production
Topic <code>serials/{serial}/check</code>	Implémenté	<code>VerificationController</code> + <code>ServeurCallback.traiterVerification()</code>
BONUS : topic status (processing/processed)	Implémenté	Commit C37 — <code>ServeurCallback.traiterCommande()</code>

8.2. Exigences non-fonctionnelles

Exigence	Statut	Où dans le code
Pas de framework Spring	Respecté	Java pur, injection manuelle dans App.java
Format de sérialisation custom (pas JSON)	Respecté	Format TYPE:qty;TYPE:qty — CommandeModel, ServeurCallback
Usine traite plusieurs commandes en parallèle	Implémenté	LinkedBlockingQueue + thread daemon + CompletableFuture
Mutualisation des commandes	Implémenté	fileDemandes.drainTo() dans boucleProduction()
Configuration externalisée (broker URL)	Implémenté	config.properties lu au démarrage
Logs pertinents (SLF4J + Logback)	Implémenté	Tous les niveaux INFO/WARN/ERROR utilisés
Gestion erreur connexion broker client	Implémenté	Dialog d'erreur fatale dans App.java client
Gestion erreur absence d'usine	Implémenté	Timeout 30s dans FabricationController
Gestion erreur fabrication	Implémenté	Try/catch → topic orders/{uuid}/error

Exigence	Statut	Où dans le code
Reconnexion automatique MQTT	Implémenté	<code>setAutomaticReconnect(true)</code> dans les deux App
Backend architecturé en Serveur + Usine	Respecté	Architecture en 4 classes respectant le schéma du sujet
Point d'entrée Usine respecte le prototype imposé	Respecté	<code>List<Lunette> produire(Map<TypeLunette, Integer> typesLunettes)</code>

8.3. Exigences de livrable

Livrable	Statut	Détail
Exécutable client prêt à l'emploi	Produit	client-win/linux/mac.zip (jlink, sans Java requis)
JAR backend prêt à l'emploi	Produit	backend-runnable.jar (fat JAR, Java 17 requis)
README d'utilisation	Livré	README.md à la racine du dépôt
Tests unitaires	Livrés	JUnit 5 + Mockito, 6 + 19 + 5 + 5 = 35 tests
CI/CD (valorisé)	Implémenté	GitHub Actions — tests + release multi-plateforme

9. Réponses aux questions du sujet

9.1. Comment gérer l'absence d'usine connectée au bus ?

Lorsque le client envoie une commande sur le topic `orders/{uuid}`, il peut arriver qu'aucun backend ne soit connecté au broker MQTT. Dans ce cas, aucun message de réponse n'est reçu et le client risquerait d'attendre indéfiniment. Pour éviter ce problème, nous avons ajouté un système de timeout dans `FabricationController`. Si aucun message validé n'est reçu après 30 secondes, un message d'erreur est affiché à l'utilisateur indiquant qu'aucune usine n'est disponible, puis les abonnements MQTT sont correctement fermés.

Une solution plus robuste pourrait utiliser le mécanisme MQTT appelé Last Will & Testament (LWT). Le backend déclarerait un message spécial auprès du broker lors de sa connexion. Si le backend se déconnecte brutalement, le broker publierait automatiquement un message indiquant que l'usine est hors ligne. Le client pourrait alors surveiller ce statut pour savoir si une usine est disponible avant même d'envoyer une commande.

Cette solution peut être améliorée grâce aux retained messages. Au démarrage, l'usine publierait un message conservé indiquant qu'elle est en ligne. Comme ce message est mémorisé par le broker, tout nouveau client connecté reçoit immédiatement l'état actuel de l'usine sans avoir besoin d'envoyer de requête supplémentaire.

9.2. Que faut-il modifier pour gérer plusieurs commandes en parallèle pour un même client ?

Actuellement, l'application cliente ne peut gérer qu'une seule commande à la fois. Un seul identifiant de commande (UUID) est suivi et l'écran `fabrication.fxml` remplace entièrement l'interface précédente. Cela signifie qu'il est impossible pour l'utilisateur de suivre plusieurs fabrications simultanément.

Pour permettre plusieurs commandes en parallèle, il faudrait d'abord ajouter une structure de données centralisée côté client, par exemple un `CommandeRegistry`, chargé de stocker l'état de toutes les commandes actives dans une map associant chaque UUID à son état. Cela permettrait de suivre plusieurs fabrications en même temps sans perdre les informations des commandes précédentes.

L'interface graphique devrait également évoluer vers un tableau de bord multi-commandes. Au lieu d'un seul écran de fabrication, l'application pourrait afficher une liste ou des onglets représentant chaque commande active. L'utilisateur pourrait alors voir en temps réel l'état de chaque commande, comme « en attente », « en cours », « livrée » ou « erreur », tout en continuant à passer de nouvelles commandes.

La gestion des abonnements MQTT devrait aussi être adaptée. Aujourd'hui, chaque `FabricationController` écoute uniquement les messages liés à un UUID précis. Avec plusieurs commandes, il serait possible soit de créer un contrôleur indépendant pour chaque commande,

soit d'utiliser un abonnement plus global afin de recevoir tous les messages puis de les redistribuer selon leur UUID dans un contrôleur centralisé.

Enfin, l'interface de navigation devrait être modifiée afin de permettre à l'utilisateur de revenir à l'écran de commande sans interrompre le suivi des fabrications déjà en cours. Une barre de navigation persistante ou un menu dédié permettrait de naviguer plus facilement entre les différentes commandes actives.

9.3. Que faut-il modifier pour pouvoir gérer plusieurs usines ?

Actuellement, le projet ne gère qu'une seule usine connectée au broker MQTT. Le backend écoute tous les messages envoyés sur `orders/+`. Si plusieurs usines étaient connectées en même temps, elles recevraient toutes les mêmes commandes et tenteraient chacune de produire les lunettes, ce qui provoquerait des doublons de fabrication.

Pour gérer plusieurs usines, une première solution consisterait à utiliser des topics distincts pour chaque usine. Chaque backend pourrait s'enregistrer avec un identifiant unique et publier ses informations, comme sa capacité de production. Le client pourrait ensuite choisir directement l'usine à laquelle envoyer la commande, par exemple avec un topic du type `orders/usine-1/{uuid}`.

Une autre amélioration possible serait d'ajouter un système de répartition automatique des commandes. Un composant dédié pourrait distribuer les commandes entre les différentes usines selon plusieurs stratégies, par exemple en alternant les usines, en choisissant celle qui a le moins de charge ou celle dont la capacité correspond le mieux à la commande demandée.

Avec MQTT 5, une solution plus propre existe grâce aux `shared subscriptions`. Plusieurs usines peuvent partager le même abonnement MQTT et le broker garantit qu'une commande ne sera envoyée qu'à une seule usine du groupe. Cela permet de répartir automatiquement les commandes sans modifier le fonctionnement du client.

Enfin, dans le cas de très grosses commandes dépassant la capacité d'une seule usine, il faudrait mettre en place une coordination entre plusieurs usines. Une usine principale pourrait par exemple répartir une partie de la production vers d'autres usines afin de traiter la commande plus efficacement.

10. Conclusion

10.1. Bilan du projet

Ce projet nous a permis de concevoir et d'implémenter un système distribué complet de bout en bout, en respectant l'ensemble des exigences du cahier des charges, y compris le bonus lié au topic status.

L'un des principaux points forts du projet est la qualité de l'architecture. La séparation claire des responsabilités entre les différentes classes (comme `Serveur`, `ServeurCallback` et `Usine`) rend le système plus lisible, plus maintenable et facilement extensible.

Le projet gère également correctement la concurrence. La classe `Usine` permet le traitement de commandes en parallèle tout en assurant un comportement cohérent, notamment grâce au découpage en lots et à l'absence de problèmes liés aux accès concurrents.

Les tests constituent également un point fort important. Avec 35 tests unitaires couvrant à la fois les cas normaux et les cas limites, ils assurent une bonne fiabilité du système et une validation solide des principales fonctionnalités métier.

La chaîne d'intégration et de déploiement continu est entièrement automatisée. Le pipeline permet de lancer les tests, de construire les artefacts et de générer des releases pour plusieurs plateformes à chaque modification du projet, garantissant un processus de livraison reproductible et fiable.

Enfin, l'expérience utilisateur a été particulièrement soignée. Des mécanismes comme le timeout, les messages d'erreur explicites et la désactivation des actions pendant les traitements permettent d'éviter les blocages et de rendre l'application plus claire et plus agréable à utiliser.

10.2. Difficultés rencontrées

La contrainte imposant de ne pas utiliser JSON pour les messages nous a obligés à concevoir notre propre protocole de sérialisation. Cela a demandé un travail supplémentaire, mais a également eu une réelle valeur pédagogique, car il a fallu réfléchir à un format simple, lisible et facilement parsable.

Le déploiement de JavaFX s'est révélé plus complexe que celui d'un simple JAR Java classique en raison de son architecture modulaire et de la présence de dépendances natives. La mise en place de la solution avec `jlink` a nécessité du temps de recherche et de configuration pour obtenir un déploiement fonctionnel sur plusieurs plateformes.

Enfin, le bug lié aux wildcards MQTT (différence entre `orders/#` et `orders/+`) illustre les subtilités propres au protocole MQTT. Ce type de problème est spécifique aux systèmes de messagerie publish/subscribe et ne se rencontre pas dans des architectures HTTP plus classiques.

10.3. Ce qui pourrait être amélioré

La persistance des commandes est actuellement entièrement en mémoire, ce qui signifie qu'une commande en cours de traitement est perdue si le backend redémarre. L'ajout d'une base de données comme PostgreSQL ou SQLite permettrait de stocker l'état des commandes et de reprendre leur traitement après un redémarrage.

L'authentification MQTT constitue également une limite importante. L'utilisation actuelle sans TLS ni authentification par identifiant et mot de passe est acceptable dans un contexte de développement local, mais insuffisante pour un environnement de production. Une amélioration prioritaire serait donc d'ajouter un chiffrement TLS ainsi qu'un système d'authentification pour sécuriser les échanges avec le broker.

Le projet pourrait aussi être enrichi par l'ajout de métriques et d'observabilité. Les logs actuels permettent de comprendre le fonctionnement de l'application, mais un système de monitoring plus avancé offrirait une meilleure visibilité sur les performances. Par exemple, il serait possible de mesurer le nombre de commandes traitées, le temps moyen de fabrication ou encore le taux d'erreur, puis d'exposer ces données via Micrometer et de les visualiser avec Grafana.

Enfin, les tests actuels sont principalement des tests unitaires. Ils ne couvrent pas l'intégration complète entre le client, le broker MQTT et le backend. L'ajout de tests d'intégration, par exemple avec un broker Mosquitto lancé via Docker Compose dans le pipeline CI, permettrait de tester le système dans des conditions plus proches de la production et d'augmenter la fiabilité globale du projet.

10.4. Ce que nous avons appris

Ce projet a été particulièrement formateur sur plusieurs aspects techniques concrets.

Il a d'abord permis de mieux comprendre MQTT et l'architecture événementielle, qui repose sur un modèle fondamentalement différent des API REST classiques. Ce paradigme introduit des concepts spécifiques comme les wildcards, les messages retained ou encore le mécanisme de Last Will Testament.

Le projet a également renforcé les compétences en programmation concurrente en Java, notamment grâce à l'utilisation de structures comme `LinkedBlockingQueue` et `CompletableFuture`. Ces outils ont permis de gérer proprement l'exécution parallèle des tâches tout en évitant les problèmes de blocage ou de conditions de concurrence.

La partie JavaFX a aussi été un apprentissage important, en particulier avec l'introduction du système modulaire de Java 9 et plus. Cette architecture a des impacts directs sur la compilation, le packaging et la distribution des applications graphiques.

Sur le plan de l'outillage, le projet a permis d'approfondir l'utilisation de Maven avec des fonctionnalités avancées comme le plugin Shade, jlink, les profils par système d'exploitation et la gestion de dépôts privés. Enfin, GitHub Actions a été utilisé pour mettre en place un pipeline

CI/CD complet, incluant des builds multi-plateformes, la gestion des secrets et la génération automatique de releases.